



Security Review For Centrifuge



Collaborative Audit Prepared For:
Lead Security Expert(s):

Centrifuge

0x52

0x73696d616f

Date Audited:

August 5 - August 15, 2026

Introduction

v3.3.0 hardens the pool-management and cross-chain surface introduced across v3.1–v3.2 and makes the protocol's core primitives more flexible. The main additions are a per-pool manifest policy layer that constrains manager actions, fully sovereign per-pool adapter sets, and per-chain risk controls that bound the damage a single compromised chain can do. This release also ships pluggable custom share tokens, a redesigned balance sheet now folded into Spoke, and flexible accounting.

Scope

Repository: centrifuge/protocol-internal

Audited Commit: f0fe9d6ef26df2984f7ab83171dcb6be4d2e6eb1

Final Commit: ac389fbc717660c26c623c5003c35a494929507b

Files:

- src/adapters/HyperlaneAdapter.sol
- src/adapters/StandbyAdapter.sol
- src/admin/GasService.sol
- src/admin/OpsGuardian.sol
- src/admin/ProtocolGuardian.sol
- src/bridge/TokenBridge.sol
- src/core/hub/Accounting.sol
- src/core/hub/Holdings.sol
- src/core/hub/HubHandler.sol
- src/core/hub/HubRegistry.sol
- src/core/hub/Hub.sol
- src/core/hub/ShareClassManager.sol
- src/core/messaging/Gateway.sol
- src/core/messaging/libraries/MessageLib.sol
- src/core/messaging/MessageDispatcher.sol
- src/core/messaging/MessageProcessor.sol
- src/core/messaging/MultiAdapter.sol
- src/core/spoke/PoolEscrow.sol
- src/core/spoke/SnapshotQueue.sol

- src/core/spoke/SpokeHandler.sol
- src/core/spoke/SpokeRegistry.sol
- src/core/spoke/Spoke.sol
- src/core/utils/BatchedMulticall.sol
- src/core/utils/Envoy.sol
- src/hooks/accounting/NAVManager.sol
- src/hooks/bridge/BridgeCircuitBreaker.sol
- src/managers/adapters/AdapterFailover.sol
- src/managers/hub/Supervisor.sol
- src/policies/hub/StdHubPolicy.sol
- src/token/ShareTokenRegistrar.sol
- src/utils/SubsidyManager.sol
- src/valuations/OracleValuation.sol
- src/vaults/AsyncRequestManager.sol
- src/vaults/BatchRequestManager.sol

Final Commit Hash

ac389fbc717660c26c623c5003c35a494929507b

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	2	8

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Pool-scoped adapters can claim another pool's deterministic share-token address [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/35>

Summary

An attacker who controls one pool's adapter quorum can submit a `NotifyShareClass` message containing another pool's salt. The destination Spoke authenticates the message using its outer `poolId`, but it does not verify that the salt belongs to that pool before passing it to the shared `ShareTokenRegistrar`.

The attacker can use the victim's salt, decimals, metadata, and the official registrar to deploy the victim's expected token address first. The victim's legitimate notification then reverts because `CREATE2` cannot deploy to the occupied address.

This also breaks the token's intended cross-network identity. The same address can represent the victim's share class on one network and an attacker-controlled pool entry on another, creating a credible token-impersonation path for users and integrations that rely on deterministic addresses.

Vulnerability Detail

When a share class is created on the Hub, `ShareClassManager` requires the first eight bytes of its salt to match the share class's `poolId`. It then records the salt as used:

[ShareClassManager.sol#L35-L57](#)

```
PoolId prefixedPoolId = PoolId.wrap(uint64(bytes8(salt)));
require(poolId == prefixedPoolId, InvalidSalt());
require(!salts[salt], AlreadyUsedSalt());

...

salts[salt] = true;
meta.salt = salt;
```

This establishes the intended salt ownership on the Hub. The destination does not repeat that validation.

`NotifyShareClass` carries `poolId`, `scId`, `decimals`, `salt`, and `registrar` as independent fields:

[MessageLib.sol#L373-L383](#)

```
struct NotifyShareClass {
    uint64 poolId;
```

```

    bytes16 scId;
    string name;
    bytes32 symbol;
    uint8 decimals;
    bytes32 salt;
    bytes32 registrar;
    uint128 extraGasLimit;
    bytes payload;
}

```

The transport routes and authenticates the message using `poolId`. It does not check whether `salt` belongs to that pool. After the message reaches `SpokeHandler.addShareClass`, the only registrar check is that its address is nonzero. The handler calls the registrar before registering the resulting token:

SpokeHandler.sol#L62-L75

```

function addShareClass(
    PoolId poolId,
    ShareClassId scId,
    string memory name,
    string memory symbol,
    uint8 decimals,
    bytes32 salt,
    IRegistrar registrar,
    bytes memory payload
) external auth {
    require(address(registrar) != address(0), InvalidRegistrar());

    address shareToken_ = registrar.newToken(name, symbol, decimals, salt, payload);
    spokeRegistry.addShareClass(poolId, scId, shareToken_, registrar);
}

```

The official `ShareTokenRegistrar` deploys tokens with the supplied salt:

ShareTokenRegistrar.sol#L112-L127

```

function newToken(
    string memory name,
    string memory symbol,
    uint8 decimals,
    bytes32 salt,
    bytes memory
) external auth returns (address) {
    ShareToken token = new ShareToken{salt: salt}(decimals);

    token.file("name", name);
    token.file("symbol", symbol);
    token.rely(root);
}

```

```

    return address(token);
}

```

The resulting address depends on the registrar address, salt, token creation code, and decimals. Neither `poolId` nor `scId` participates in the derivation:

ShareTokenRegistrar.sol#L193-L210

```

bytes32 hash = keccak256(
    abi.encodePacked(
        bytes1(0xff),
        address(this),
        salt,
        keccak256(abi.encodePacked(type(ShareToken).creationCode,
            ↪ abi.encode(decimals)))
    )
);

```

The protocol's deployment scripts place `ShareTokenRegistrar` at a deterministic address using `CREATE3`:

FullDeployer.s.sol#L308-L314

```

shareTokenRegistrar = ShareTokenRegistrar(
    create3(
        createSalt("shareTokenRegistrar", V3_3),
        abi.encodePacked(type(ShareTokenRegistrar).creationCode, abi.encode(root,
            ↪ batcher))
    )
);

```

The deployment scripts use the same mainnet deployer and suffix across networks. As a result, the registrar and the tokens it deploys can have the same addresses on each supported network. Reusing a share class's salt and decimals therefore produces its expected cross-network token address.

An attacker can exploit this as follows:

1. A victim creates a share class on the Hub. Its salt, decimals, name, and symbol become public before or while its notification is delivered to another chain.
2. The attacker controls a separate pool and its adapter quorum on the destination chain.
3. The attacker submits a `NotifyShareClass` message routed under the attacker's `poolId`, but supplies the victim's salt, decimals, metadata, and the official `ShareTokenRegistrar`.
4. `SpokeHandler` calls the official registrar, which deploys an official `ShareToken` at the address reserved for the victim's share class.

5. SpokeRegistry records that address under the attacker's (poolId, scId):

SpokeRegistry.sol#L85-L101

```
require(tokenDetails[shareToken_].poolId.isNull(), TokenAlreadyRegistered());

shareClass_.shareToken = IERC20(shareToken_);
shareClass_.registrar = registrar_;
tokenDetails[shareToken_] = TokenDetails(poolId, scId);
```

6. When the victim's legitimate notification arrives, the registrar attempts the same CREATE2 deployment and reverts because the address already contains code.

The attacker can also grant themselves the Spoke manager role for their pool through a forged UpdateManager message:

MessageProcessor.sol#L210-L222

```
if (managerKind == ManagerKind.Spoke) {
    spokeHandler.updateManager(poolId, who, m.canManage);
}
```

Once authorized as a manager, the attacker can issue balances of the predeployed token through Spoke.issue:

Spoke.sol#L231-L239

```
function issue(PoolId poolId, ShareClassId scId, address to, uint128 shares)
    external
    payable
    enforced(poolId)
{
    (IERC20 token, IRegistrar registrar) =
        spokeRegistry.shareTokenAndRegistrar(poolId, scId);

    snapshotQueue.queueShares(poolId, scId, shares, true);
    registrar.mint(address(token), to, shares);
}
```

The attacker does not need to authenticate as the victim's pool. Authentication under the attacker's pool is enough because the salt reaches a shared CREATE2 namespace without being rebound to the authenticated poolId.

Impact

The victim's share class cannot be deployed on the affected Spoke through the normal notification flow. Its committed salt resolves to an address that already contains code, causing every legitimate deployment attempt to revert.

The collision also breaks the intended cross-network identity of the share token. On unaffected networks, the deterministic address represents the victim's share class. On the affected network, the same address can be registered under the attacker's pool, display the victim's name and symbol, and contain balances issued through the protocol's own Spoke and registrar.

Wallets, token lists, frontends, bridges, and other integrations that copy address allowlists across networks or treat the deterministic address as sufficient proof of identity may display or accept those balances as the victim's shares.

This creates a token-impersonation path and may cause users or integrations to accept balances that are not shares of the victim's pool. Recovery requires privileged intervention or a different deployment scheme. Reassigning the occupied token to the victim is unsafe if the attacker has already issued balances or changed its local configuration.

Code Snippet

[ShareClassManager.sol#L35-L57](#)

[SpokeHandler.sol#L62-L75](#)

[ShareTokenRegistrar.sol#L112-L127](#)

[ShareTokenRegistrar.sol#L193-L210](#)

[SpokeRegistry.sol#L85-L101](#)

Tool Used

Manual Review

Recommendation

Validate the salt's pool prefix in `SpokeHandler.addShareClass` before calling the registrar:

```
PoolId saltPoolId = PoolId.wrap(uint64(bytes8(salt)));  
require(saltPoolId == poolId, InvalidSalt());
```

Discussion

hieronx

Fixed in <https://github.com/centrifuge/protocol-internal/pull/351>

Issue M-2: A frozen investor is able to extract escrowed shares via the shares claim paths [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/37>

Summary

Shares leave the pool escrow through `Spoke::withdrawShares()`, which leads to `from = poolEscrow` and `to = receiver` and the controller is never checked, so a frozen investor is able to extract its escrowed shares.

Vulnerability Detail

`AsyncVault::deposit()` does `shares = asyncManager.deposit(this, assets, receiver, controller);`, and following the call path never checks for `_canTransfer()`.

However, `AsyncRequestManager::maxDeposit()` and `AsyncRequestManager::maxMint()` both call `AsyncRequestManager::_canTransfer()` on exactly the escrow to controller edge and return 0 for a frozen controller, and `AsyncRequestManager::_processRedeem()` and `AsyncRequestManager::claimCancelDepositRequest()` require the controller to pass that same check before releasing assets.

Impact

A frozen investor extracts their whole `maxMint` and `claimableCancelRedeemRequest` to an address they pick: any non frozen one under `FreezeOnly::checkERC20Transfer()` and `RedemptionRestrictions::checkERC20Transfer()` or any member address under `FullRestrictions::checkERC20Transfer()`. An expired memberlist entry escapes the same way, while `AsyncRequestManager::maxMint()` reports 0 to every ERC-4626 integrator.

Code Snippet

`AsyncRequestManager::_processDeposit()`, which delegates the restriction checks to the share transfer:

```
// NOTE: Assumes restrictions check of receiver to be done in the share transfer
spoke.withdrawShares(vaultDetails.poolId, vaultDetails.scId, receiver, sharesDown);
```

Against `AsyncRequestManager::_processRedeem()`, which does check the controller:

```
if (controller != receiver) {
    require(
```

```
        _canTransfer(vault_, controller, receiver, convertToShares(vault_,  
        ↪ assetsDown)), TransferNotAllowed()  
    );  
}
```

Tool Used

Manual Review

Recommendation

Require AsyncRequestManager::_canTransfer() on the escrow to controller edge in AsyncRequestManager::_processDeposit() and AsyncRequestManager::claimCancelRedeemRequest(), just like AsyncRequestManager::maxMint().

Discussion

hieronx

Fixed in https://github.com/centrifuge/protocol-internal/pull/360

Issue L-1: StandbyAdapter does not account for its additional execution cost and EIP-150 gas reduction [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/36>

Summary

`StandbyAdapter.forward` passes the original `gasLimit` to the underlying adapter without accounting for the additional `StandbyAdapter.handle` execution layer. A normal delivery follows `Adapter → MultiAdapter`, while a standby delivery follows `Adapter → StandbyAdapter → MultiAdapter`.

The standby path adds approximately 3,000 to 4,000 gas of execution cost based on testing, along with another EIP-150 63/64 gas reduction when `StandbyAdapter` calls `MultiAdapter`. For a message requiring 100,000 gas, the additional EIP-150 boundary removes roughly 1,563 gas. Combined with the wrapper's execution cost, `MultiAdapter` may receive around 4.5% to 5.5% less gas than expected.

This can cause a standby delivery to run out of gas even though the same `gasLimit` is sufficient through a normal adapter. Since the standby adapter is intended to restore liveness when another adapter is unavailable, underfunding this path can prevent it from supplying the missing quorum vote.

Code Snippet

[StandbyAdapter.sol#L62-L71](#)

[StandbyAdapter.sol#L80-L88](#)

Tool Used

Manual Review

Recommendation

Increase the gas limit passed to `underlying.estimate` and `underlying.send` by a fixed reserve for the execution cost of `StandbyAdapter.handle`, then scale it to compensate for the additional EIP-150 boundary.

For example:

```
uint256 adjustedGasLimit =  
    Math.ceilDiv((gasLimit + STANDBY_RECEIVE_COST) * 64, 63);
```

Discussion

hieronx

Fixed in <https://github.com/centrifuge/protocol-internal/pull/351>

Issue L-2: SyncDepositVault calls its ERC-4626 conversion views from the async redeem manager, which is wrong on a sync vault [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/38>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

SyncDepositVault passes the async redeem manager into BaseVault as BaseVault::baseManager, so the four conversion views quote the registry share price, while the deposit uses SyncManager::pricePoolPerShare(), returning the pool's configured valuation instead.

Vulnerability Detail

BaseVault::totalAssets(), BaseVault::convertToShares(), BaseVault::convertToAssets() and BaseVault::pricePerShare() are not virtual, so they land in AsyncRequestManager::convertToShares() and AsyncRequestManager::convertToAssets(), which read SpokeRegistry::pricePoolPerShare().

BaseSyncDepositVault::deposit() and BaseSyncDepositVault::previewDeposit() instead reach SyncManager::convertToShares(), and BaseSyncDepositVault::mint() and BaseSyncDepositVault::previewMint() reach SyncManager::_shareToAssetAmount():

```
if (address(valuation_) == address(0)) {
    price = spokeRegistry.pricePoolPerShare(poolId, scId, true);
} else {
    price = valuation_.pricePoolPerShare(poolId, scId);
}
```

Impact

An integrator calling BaseVault::convertToShares(), or an accounting system valuing a position from BaseVault::pricePerShare(), get the spoke registry price while the deposit executes at the valuation price.

Code Snippet

SyncDepositVault:

```
BaseVault(poolId_, scId_, asset_, token_, root_,  
↳ IBaseRequestManager(address(asyncRedeemManager_)))  
BaseSyncDepositVault(syncDepositManager_)  
BaseAsyncRedeemVault(asyncRedeemManager_)
```

Tool Used

Manual Review

Recommendation

Make the four views virtual on BaseVault and override them in BaseSyncDepositVault to price through BaseSyncDepositVault::syncDepositManager.

Discussion

hieronx

Acknowledged but since it predates this release, don't think it's worth fixing. We will do a full vault redesign soon and will address it there.

Issue L-3: Supervisor::_checkNotSelfRemoval() reverts on a malformed target instead of returning [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/39>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Supervisor::_checkNotSelfRemoval() casts managerCall using CastLib::toAddress(), which reverts on any word whose low 12 bytes are non zero, so a sentinel vetoing such an authorization reverts.

Vulnerability Detail

Supervisor::cancelAuthorization() calls Supervisor::_checkNotSelfRemoval() whenever more than one sentinel is registered. The guard is a chain of early returns: a foreign selector, args under 224 bytes, a short inner payload and a non RemoveSentinel payload all return and leave the veto intact. However, the target comparison runs CastLib::toAddress(), which reverts for bytes32 having low non zero 12 bytes.

It's reachable because StdHubPolicy::_checkManagerCall() returns delay for a remote centrifugeId before it casts the target.

Impact

The sentinel set loses its veto over that authorization, and the entry sits in HubRegistry::authorizedAfter until a pool manager clears it. Hub::managerCall() does the same, so the authorization is impossible too.

Code Snippet

Supervisor::_checkNotSelfRemoval() reverts on a malformed target:

```
if (target.toAddress() != address(this)) return;
if (payload.length < 64) return;
if (payload.toUint256(0) != uint256(uint8(TrustedCall.RemoveSentinel))) return;
```

CastLib::toAddress():

```
require(uint96(uint256(addr)) == 0, PrefixNotZero());
```

Tool Used

Manual Review

Recommendation

Compare and return early instead.

Discussion

hieronx

Acknowledged, no impact.

Issue L-4: SpokeHandler::executeTransferShares() only validates the receiver on the destination chain [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/40>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`receiver` is only checked on the destination chain, after the origin has already burned the shares, so a right aligned `bytes32` receiver will keep reverting,

Vulnerability Detail

`SpokeHandler::executeTransferShares()` decodes the using `CastLib::toAddress()`, which requires the low 12 bytes to be zero, the form `CastLib::toBytes32()` produces. A right aligned word reverts there with `CastLib::PrefixNotZero()`. However, nothing on the origin checks that.

Impact

The sender loses the whole bridged amount, burned on the origin and unmintable on the destination, with `HubHandler::initiateTransferShares()` having already moved the issuance to the target chain.

Code Snippet

`Spoke::crosschainTransferShares()` burns without checking `receiver`:

```
SafeTransferLib.safeTransferFrom(address(share), owner, address(this), amount);
SafeTransferLib.safeApprove(address(share), address(registrar), amount);
registrar.burn(address(share), address(this), amount);
```

Tool Used

Manual Review

Recommendation

Validate the encoding on the origin.

Discussion

hieronx

This is infeasible to fix: in the future a bridge transaction might go to a non-EVM spoke.
This is listed in out of scope.

Issue L-5: Hub::setAccountMetadata() and Hub::setHolding are unreachable on onchainAccounting pools [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/42>

Summary

StdHubPolicy::_isAccountingSelector() includes Hub::setAccountMetadata() and Hub::setHoldingAccountId() and when under StdHubPolicy::onchainAccounting are gated to NAVManager, which doesn't implement them.

Vulnerability Detail

Under StdHubPolicy::onchainAccounting, an accounting selector requires the caller to be StdHubPolicy::navManager, reverting otherwise. NAVManager calls Hub only to create accounts, initialize a holding, update its value and valuation and add a journal, so the other two selectors have no in-policy caller.

However, they are still reachable through the Hub, since Accounting::setAccountMetadata() and Holdings::setAccountId() are auth, so they can still be called through other ways.

Impact

On a pool with StdHubPolicy::onchainAccounting, account metadata cannot be set and a holding's account ids cannot be repointed except by another auth authority.

Code Snippet

StdHubPolicy::_authorizationDelay()

```
if (_isAccountingSelector(selector)) {  
    require(caller == navManager, OnchainAccountingOnly());  
    return 0;  
}
```

StdHubPolicy::_isAccountingSelector()

Tool Used

Manual Review

Recommendation

Document in StdHubPolicy::_isAccountingSelector() that Hub::setAccountMetadata() and Hub::setHoldingAccountId() are intentionally frozen under StdHubPolicy::onchainAccounting and reachable only through another auth.

Discussion

hieronx

setAccountMetadata is fixed in <https://github.com/centrifuge/protocol-internal/pull/328>

setHoldingAccountId is intentionally not exposed.

Issue L-6: Supervisor::cancelAuthorization() skips its guard at one sentinel, which also means the last sentinel can veto its removal forever [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/45>

Summary

Supervisor::fromHub() can't remove the last sentinel, so a pool holding one has to add a second first, which is an out of policy managerCall that goes through a timelock, and Supervisor::cancelAuthorization() can be called repeatedly so the sentinel stays there forever.

Vulnerability Detail

The removal branch of Supervisor::fromHub() requires Supervisor::sentinelCount above one. The add is reachable only through Hub::managerCall(), and StdHubPolicy::_checkManagerCall() doesn't check for the Supervisor, so it goes through a delay

The cancel has no delay as seen in StdHubPolicy::_authorizationDelay() and HubRegistry::cancelAuthorization() deletes the attempt.

Impact

A pool with one sentinel cannot replace it, unless Hub::setPolicy() is called by a ward.

Code Snippet

Supervisor::cancelAuthorization() always leaves at least 1 sentinel.

```
function cancelAuthorization(bytes calldata data) external onlySentinel {
    if (sentinelCount > 1) {
        _checkNotSelfRemoval(data, msg.sender);
    }
    hub.cancelAuthorization(poolId, data);
}
```

Supervisor::fromHub() doesn't allow removing the last sentinel.

```
require(sentinels[sentinel], NotSentinel());
require(sentinelCount > 1, LastSentinel());
```

Tool Used

Manual Review

Recommendation

Document this behaviour and explain the recovery.

Discussion

hieronx

Agree we should document this.

Issue L-7: SyncManager::maxDeposit() and SyncManager::maxMint() revert with InvalidPrice while a price is still unset, instead of returning 0 [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/46>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

SyncManager::maxDeposit() and SyncManager::maxMint() call SpokeRegistry::pricePoolPerAsset() which reverts with InvalidPrice while computedAt is 0, but IERC7575 requires these views to never revert.

Vulnerability Detail

SyncManager::maxDeposit() computes the reserve headroom with SyncManager::_maxDeposit(), returning 0 for an unlinked vault, then calls SyncManager::_maxAssetsAndShares(), which reverts if pricePoolPerAsset or pricePoolPerShare are not valid.

Impact

BaseSyncDepositVault::maxDeposit() and BaseSyncDepositVault::maxMint() revert instead of following the ERC-4626 and ERC-7575 specification.

Code Snippet

SyncManager::_maxConvertibleAssets(), reached before maxAssets is looked at:

```
D18 poolPerShare = pricePoolPerShare(vaultDetails.poolId, vaultDetails.scId);
D18 poolPerAsset =
    spokeRegistry.pricePoolPerAsset(vaultDetails.poolId, vaultDetails.scId,
    ↪ vaultDetails.assetId, true);

if (poolPerShare.isZero() || poolPerAsset.isZero()) return 0;
```

SpokeRegistry::pricePoolPerAsset():

```
Price memory poolPerAsset = _pricePoolPerAsset[poolId][scId][assetId];
require(!checkValidity || poolPerAsset.isValid(), InvalidPrice());
```

Tool Used

Manual Review

Recommendation

Use `checkValidity = false` inside `SyncManager::_maxConvertibleAssets()`, `SyncManager::convertToShares()` and `SyncManager::_shareToAssetAmount()`, just like `AsyncRequestManager::convertToShares()`, but make sure to keep the `revert` in `SyncManager::deposit()` and the state changing functions.

Discussion

hieronx

Acknowledged but since it predates this release, don't think it's worth fixing. We will do a full vault redesign soon and will address it there.

Issue L-8: Protocol deployment scripts do not use the scoped DeployGate revision [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-08-centrifuge-august-5th-2026/issues/50>

Summary

The scoped `create3-gate` revision is `41e0f70`, but `protocol-internal` pins its `lib/create3-gate` submodule to the older ancestor `0665d63`. Because the protocol remaps `create3-gate/` imports to this submodule, `GatedDeployer` compiles against the older gate and deploys it at `0xBA08f1fD092031C81296fC59f6539b21b38f2249`, rather than deploying the scoped gate at `0xac9C69450016396F0A00648aBA4488433f958E12`. The gate address is also part of every downstream `CREATE3` address derivation, so the production deployment generated by the current scripts will not correspond to the scoped gate revision or its derived addresses.

Vulnerability Detail

Impact

Code Snippet

The protocol redirects `create3-gate/` imports to its pinned submodule:

[remappings.txt#L15-L16](#)

```
create3-gate/=lib/create3-gate/
```

`GatedDeployer` consequently imports its gate address from that dependency:

[GatedDeployer.s.sol#L8-L13](#)

```
import {DEPLOY_GATE_ADDRESS} from "create3-gate/script/DeployGate.d.sol";
```

The pinned `0665d63` dependency uses `0xBA08...`:

[DeployGate.d.sol@0665d63#L9-L14](#)

```
address constant DEPLOY_GATE_ADDRESS = 0xBA08f1fD092031C81296fC59f6539b21b38f2249;
```

The scoped `41e0f70` revision instead uses `0xac9C...`:

[DeployGate.d.sol@41e0f70#L9-L14](#)

```
address constant DEPLOY_GATE_ADDRESS = 0xac9C69450016396F0A00648aBA4488433f958E12;
```

Tool Used

Manual Review

Recommendation

Update the protocol-internal/lib/create3-gate submodule to 41e0f70

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.